

计算机网络大题专项：CRC、路由、IP、拥塞控制、子网与分片

这份文档的定位： 从零基础开始，先建立直觉理解，再讲做题方法。每个章节按「大白话 → 概念拆解 → 做题步骤 → 例题 → 易错点」组织。

阅读建议： 如果时间紧，至少把每个章节的「大白话理解」和「直觉类比」读完，这两部分帮你建立心智模型。纯背公式而没有心智模型，题目稍微变形就会卡住。

零基础前置：先搞懂这些再往下看

在进入具体计算题之前，需要先建立几个核心概念。这些概念就像盖房子前的地基——没有它们，后面的公式只是无意义的符号。

前置1：数据在网络中传输，会出错

发送方

————[电线/光纤/无线]————→

接收方

↑

电流干扰、信号衰减、噪声

都可能导致 0 变成 1，1 变成 0

直觉类比： 你在嘈杂的餐厅里跟朋友说话，朋友听到的可能和你说的不一样。"我要红烧牛肉面" → 朋友听到"我要红烧牛叉面"。计算机网络面临同样的问题——数据传着传着就可能出错。

CRC 就是用来发现这种错误的——就像在包裹外面贴一张"物品清单"，收件人拆包后核对清单，不一致就说明运输中出了问题。

前置2：IP地址 = 互联网世界的"门牌号"

现实中快递：

中国 → 江西省 → 南昌市 → 某小区 → 某栋 → 某户

网络中数据：

网络号（定位到哪个网络）→ 主机号（定位到哪台机器）

IP地址本质上是一个32位的二进制数，人为切成两段：

- 前半段（网络号）： 相当于"江西省南昌市"——定位到一个具体的网络
- 后半段（主机号）： 相当于"某小区某栋某户"——定位到网络里的具体设备

子网掩码就是告诉你"从哪里切开"——/24 意思是"前24位是网络号，后8位是主机号"。

前置3：路由器 = 快递中转站

你 → 路由器A → 路由器B → 路由器C → 目标服务器

"这个包去北京？"
往北走"

"这个包去北京方向"

"这个包去北京，到了"

路由器的工作就一件事：**收到一个数据包，看目的地址，决定下一站发给谁。** 它不关心数据包里装的是什么，只关心"往哪送"。

- **路由表 = 快递公司的分拣规则手册** ("去北京的走3号传送带，去上海的走5号")
- **路由协议（RIP等）= 各中转站互相通信"我这边到XX有多远"**，这样大家都能更新自己的路线图

前置4：网络有"堵车"问题

早高峰： 1000辆车同时涌向一个收费站 → 堵死

网络中： 1000个数据包同时涌向一个路由器 → 缓冲区满 → 丢包

拥塞控制就是TCP的"智能导航"——发现前方堵车就主动减速，路况好再慢慢加速。不是等到撞车了才刹车，而是通过"有没有收到确认回执"来推断网络是否拥堵。

前置5：数据包不能无限大

搬家： 一整屋家具 → 太大了，一辆面包车装不下 → 拆成几车运

网络中： 一个大数据包 → 超过了链路的最大传输单元（MTU） → 拆成几个小包

IP分片就是"把大包裹拆成小包裹"，每个小包裹上标注"我是第几片"和"后面还有没有"。

搞清楚了这五个前置概念，下面的每一章就都好理解了。你本质上是在学：

章节	要解决的问题	核心思想
CRC	数据传错了怎么办？	发方算一个"校验码"贴在后面，收方重算一遍对比
子网掩码	怎么知道IP的"网络号"和"主机号"？	掩码告诉你从第几位切开
路由聚合	路由表太长了怎么办？	把相邻的地址块合并成一个更大的块

章节	要解决的问题	核心思想
IP转发	路由器收到包往哪发？	查路由表，找匹配得最精确的那条
RIP	路由器怎么知道去每个网络走哪最近？	邻居之间互相交换"距离表"，谁更近就更新
IP分片	大包过不了小链路怎么办？	拆成小包，每个标注序号
拥塞控制	网络堵了怎么办？	通过丢包信号推断拥堵，主动调速

1. CRC 循环冗余校验

1.0 大白话理解

CRC解决什么问题？ 数据在传输过程中可能出错（0变1、1变0），接收方需要一种方法**发现**这些错误。

核心思想一句话： 把数据当成一个巨大的二进制数，用另一个固定的二进制数（生成多项式）去除它，把除出来的余数贴在数据后面一起发送。接收方收到后用同样的数去除——如果余数是0，大概率没出错；如果不为0，一定出错了。

直觉类比： 你给朋友快递100本书，你在包裹里放一张纸条"总共100本"。朋友收到后数一遍——如果是100本，大概率运输没出问题；如果是97本，肯定中间丢了。**CRC余数就是这张"数字纸条"**。

发送方：数据=10111110，除以生成多项式1001，余数=100
→ 发送 10111110 + 100 = 10111110100

接收方：收到 10111110100，同样除以1001
→ 余数=0？数据大概率完整 ✓
→ 余数≠0？数据一定出错了 X（请求重发）

1.1 核心概念拆解

概念1：生成多项式 G(x) 是什么？

就是一个你事先选好的"除数"。它被写成多项式形式，但实际计算时转成二进制串。

$x^3 + 1$

→ 二进制 1001

(x^3 对应1, x^2/x^1 没有对应0, 常数1对应1)

$x^3 + x + 1$

→ 二进制 1011

(x^3 有、 x^2 没有、 x^1 有、常数1有)

$x^4 + x^2 + 1$

→ 二进制 10101

怎么写二进制串：从最高次到0次，系数为1就写1，系数为0就写0。最高次一定是1。

概念2：模2除法是什么？

就是二进制的异或（XOR）除法——关键差异在于"减"这一步：没有借位，直接逐位XOR。

普通减法：1-1=0，0-0=0，1-0=1，0-1需要借位

模2减法：相同=0，不同=1（本质上就是XOR）

模2除法的具体操作（手算时这样写）：

被除数：1011110000

除数：1001

- ① 除数对齐被除数最左边的1
- ② 逐位XOR
- ③ 把结果中左边的0去掉，拉下被除数的下一位
- ④ 重复直到所有位处理完
- ⑤ 最后剩下的就是余数

概念3：为什么是"校验"而不是"纠错"？

CRC只能发现错误，不能自动修正错误。发现错误后，通常的做法是让发送方重传。这是工程上的取舍——纠错码需要更多冗余信息（占用更多带宽），而重传在大多数情况下更经济。

1.2 做题步骤

1. 从生成多项式提取二进制串（也是确定最高次 r ）
2. 在原始数据后面补 r 个 0
3. 用生成多项式对应的二进制串做模2除法
4. 得到 r 位余数（不够 r 位时前面补0）
5. 发送比特串 = 原始数据 + CRC余数

每一步的"Why":

- 补 r 个 0：相当于把数据"左移 r 位"，给余数腾位置
- 模2除法：XOR操作在硬件上用几个门电路就能实现，极快

1.3 例题

题目：

数据 $M = 1011110$ ，生成多项式 $P(x) = x^3 + 1$ ，求CRC余数和发送比特串。

一步步解：

Step 1: $P(x) = x^3 + 1 \rightarrow$ 二进制串 1001 ，最高次 $r = 3$

Step 2: 在 M 后面补 3 个 0 $\rightarrow 1011110000$

Step 3: 模2除法 $1011110000 \div 1001$

详细除法过程（跟着走一遍）：

```
1011110000
1001          ← 对齐最高位的1, XOR
-----
010110000    ← 去掉前面的0
1001          ← 再次对齐, XOR
-----
00101000
1001          ← 对齐, XOR
-----
0001100
1001          ← 对齐, XOR
-----
01010
1001          ← 对齐, XOR
-----
0011 → 余数, 不够3位前面补0 → 100
```

等一下，让我重新仔细算这一步...

实际上手工算的时候更简单：

```
1011110000
÷ 1001
-----
= 0110110 余 100
```

（这里的关键是：每次用除数的最高位1"消掉"被除数当前的最高位1，消完后剩下的位数少于除数，就是余数）

Step 4: 余数 = 100 （恰好3位，不用补）

Step 5: 发送比特串 = 原始数据 + CRC余数 = $1011110\ 100 = 1011110100$

答案：

CRC余数 = 100
发送比特串 = 1011110100

1.4 常见陷阱

陷阱1：生成多项式最高次是几，就补几个0——不是看多项式里有几项。
 x^3+1 最高次是3，补3个0。不是因为2项就补2个0。

陷阱2：余数不够 r 位时，必须在前面补0。
比如 $r=3$ 但余数只有 10，要写成 010。

陷阱3：CRC用于差错检测，不是自动纠错。发现错了需要重传。

2. IPv4地址与子网掩码

2.0 大白话理解

IP地址就是互联网上每台设备的"身份证号"。它长这样：192.168.1.100，本质上是一个32位的二进制数，为了人类可读，每8位转成一个十进制数，中间用点隔开。

为什么要分网络号和主机号？想象中国邮政系统——如果每个地址只是"第1号住户、第2号住户..."一路编到14亿，邮局根本无法高效分拣。实际做法是"省→市→区→街道→号"，层级化之后每级只管理自己那一层。IP地址的网络号/主机号拆分，就是同样的道理。

直觉类比：电话号码。0571-88201234——区号0571告诉你"在杭州"，后面的88201234是杭州内部的具体号码。IP地址 = 网络号（区号）+ 主机号（本地号码）。子网掩码就是在说"区号占几位"。

2.0.0 IPv4地址分类（A/B/C/D/E）

先有"分类编址"这套老方案，后来发现太浪费才引出了CIDR（无类别编址，也就是你常看到的/24 /26 这种写法）。但考试还会考分类，所以需要记。

类别	第一个字节	网络号位数	主机号位数	默认掩码	适用场景
A	1~126	8位	24位	255.0.0.0 (/8)	大型网络（国家骨干网级别）
B	128~191	16位	16位	255.255.0.0 (/16)	中型网络（大企业/高校）

类别	第一个字节	网络号位数	主机号位数	默认掩码	适用场景
C	192~223	24位	8位	255.255.255.0 (/24)	小型网络（家庭/小公司）
D	224~239	不区分	不区分	无	组播（一对多发送）
E	240~255	不区分	不区分	无	保留实验用

记忆技巧——看第一个数字就行：

0~127	→ A类（0和127有特殊用途，实际可用1~126）
128~191	→ B类（128是B的起点，192是C的起点）
192~223	→ C类（192.168.x.x 是最常见的家庭路由器地址）
224~239	→ D类（组播，考试知道是D类就行）
240~255	→ E类（保留，考试几乎不出）

特殊IP地址（考试高频）

地址	含义	一句话记
127.0.0.1	回环地址	"我发给自己的测试信"——数据不离开本机
0.0.0.0	本网络本主机	"还没拿到正式身份证"——DHCP分配IP前使用
255.255.255.255	受限广播	"在本房间大喊一声"——路由器不转发，只在本网内
网络号.全0	表示本网络	如192.168.1.0表示"192.168.1这个网段"
网络号.全1	直接广播	如192.168.1.255表示"向192.168.1网段所有人广播"

私有地址（内网地址）

为什么要有私有地址？IPv4地址不够用了，内网用私有地址，公网共用少数公网IP。 路由器看到私有地址不会转发到公网——就像公司内线电话不能直接打外线。	
A类私有：10.0.0.0 ~ 10.255.255.255	（10.x.x.x 打头的一定是内网）
B类私有：172.16.0.0 ~ 172.31.255.255	（172.16~31，注意不是全部172）
C类私有：192.168.0.0 ~ 192.168.255.255	（192.168.x.x 打头的一定是内网）

2.0.1 IPv4二进制 ↔ 点分十进制

二进制 → 十进制（图片题常考）

IPv4地址是32位，分4组，每组8位。8位权值从高到低固定为：

128	64	32	16	8	4	2	1
↑					↑	↑	
2 ⁷					2 ¹	2 ⁰	

算的时候只把1对应的权值加起来就行：

```
00001010 = 8 + 2 = 10
11111110 = 128+64+32+16+8+4+2+0 = 254
00001111 = 8+4+2+1 = 15
11110000 = 128+64+32+16 = 240
```

★ Insight

1. 11111111 = 255 是8位全1（128+64+32+16+8+4+2+1），这是为什么子网掩码里总是看到255——它就是"这一组8位全归网络号"的意思。
2. 权值表不用死记，记住128开头然后每次÷2就行：128→64→32→16→8→4→2→1
3. 看到00001010不要一位位加，直接认：8+2=10。多练几遍就条件反射了。

十进制 → 二进制

方法1（除2取余）：适合笔算，准确但慢。

```
130 ÷ 2 = 65 余 0
65 ÷ 2 = 32 余 1
32 ÷ 2 = 16 余 0
16 ÷ 2 = 8 余 0
8 ÷ 2 = 4 余 0
4 ÷ 2 = 2 余 0
2 ÷ 2 = 1 余 0
1 ÷ 2 = 0 余 1
```

从下往上读余数 → 10000010

方法2（权值凑数，推荐考试用）：心算快。

```
130 = 128 + 2
→ 128那位置写1，2那位置写1，其余位置写0
→ 10000010
```

重要！不足8位时前面补0。比如 10 的二进制是 1010，但IPv4里必须写成 00001010。

2.0.2 特殊地址：源/目的地址判断

题目常问："哪个地址只能作源IP地址，不能作目的IP地址？"

地址	能作源?	能作目的?	理由
0.0.0.0	✓	✗	"我还没有正式地址", DHCP获取IP时用
0.host-id	✓	✗	指名本网络上的某台主机
255.255.255.255	✗	✓	受限广播, "向所有人喊", 不能作为"来自"地址
net-id.全1	✗	✓	向指定网络广播
127.x.x.x	✓	✓	回环, 本机内部测试
普通单播地址	✓	✓	正常通信

2.0.3 分类编址：按拓扑分配IP地址

这类题在考什么？ 给一张网络拓扑图，让你给各个局域网和链路分配IP地址段。本质是"数一数需要多少地址，选合适的网络类别"。

做题步骤：

1. 先数有几个网络：路由器接口分隔网络，交换机/集线器不分隔。
(规则：过路由器 = 新网络，过交换机 = 同一网络)
 2. 看每个网络需要多少主机地址（路由器接口也要占一个地址）。
 3. 选类别：A类≈1677万主机，B类≈6.5万，C类≈254。
选刚好能装下的最小类别。
 4. 给每个网络分配不同的网络号，再分配具体接口/主机地址。

分类编址下可用主机数速查：

类别	可用主机数	判断标准
A类	$2^{24} - 2 \approx 1677$ 万	需要超过65534台
B类	$2^{16} - 2 = 65534$	需要255~65534台
C类	$2^8 - 2 = 254$	需要不超过254台

注意： -2是因为网络地址（全0主机号）和广播地址（全1主机号）不能分配给具体设备。

2.1 子网掩码核心公式

先建立直觉

IP地址：192.168.1.130

把它拆开：192.168.1.130

└─网络号─┐└─主机号─┐

(到哪一栋) (哪个房间)

子网掩码 /26 的意思是：前26位是网络号，后6位是主机号。

那怎么知道第26位在哪呢？

26 = 8 + 8 + 8 + 2 → 前三组用满 + 第四组的前2位

第四组还剩 8-2 = 6 位给主机 → $2^6 = 64$ 个地址一组

必背公式

主机位数 $h = 32 - \text{前缀长度}$

地址总数 = 2^h

可用主机数 = $2^h - 2$ (减掉网络地址和广播地址)

网络地址：主机位全是0

广播地址：主机位全是1

块大小 = 2^h (每个子网占多少个地址)

= 256 - 掩码中非255的字节

常见掩码速查表（考试必背）

前缀	子网掩码	块大小	可用主机数
/24	255.255.255.0	—	254
/25	255.255.255.128	128	126
/26	255.255.255.192	64	62
/27	255.255.255.224	32	30
/28	255.255.255.240	16	14
/29	255.255.255.248	8	6
/30	255.255.255.252	4	2

理解为什么是这个数：以 /26 为例，掩码 255.255.255.192。

192 = 128 + 64 → 第四字节前2位是1 → 后6位是0 → $2^6 = 64$ 个地址一组 → 块大小=64。

★ Insight

掩码 255.255.255.192 为什么块大小是 64?

192 的二进制 = 11000000, 前面2个1, 后面6个0。

后6位可以表示 0~63 (共64个数), 所以每个子网有64个地址。

这就是"块大小"的物理含义——主机位能表示的地址范围。

2.2 例题：求网络号与广播地址

题目：

IP地址：192.168.1.130/26

求网络地址、广播地址、可用主机范围。

解法：块大小法（最快，推荐考试用）

Step 1: /26 掩码 = 255.255.255.192, 块大小 = $256 - 192 = 64$

Step 2: 划分子网范围（每64个一组）：

子网0: 192.168.1.0 ~ 192.168.1.63

子网1: 192.168.1.64 ~ 192.168.1.127

子网2: 192.168.1.128 ~ 192.168.1.191 ← 130 在这里

子网3: 192.168.1.192 ~ 192.168.1.255

Step 3: 确定答案：

130 落在 128~191 这个子网里

答案：

网络地址: 192.168.1.128 (子网的第一个地址, 主机位全0)

广播地址: 192.168.1.191 (子网的最后一个地址, 主机位全1)

可用主机: 192.168.1.129 ~ 192.168.1.190

可用主机数: $64 - 2 = 62$ (总数减掉网络地址和广播地址)

3. 路由聚合

3.0 大白话理解

路由聚合解决什么问题？互联网上的路由器有海量路由表条目，查表很慢。如果能把"去192.168.0.0/24、192.168.1.0/24、192.168.2.0/24、192.168.3.0/24"这4条合成1条"去

192.168.0.0/22"，路由表就从4条变1条。

直觉类比： 快递分拣。不需要每个城市都单独编号——"所有33开头的邮政编码都是去浙江省的"。33就是前缀，后面的数字再细分城市。路由聚合就是找"所有去往这些网络的包，它们的地址前面几位都是一样的，可以合并"。

3.1 核心思想

把多个连续的小地址块 → 合并成一个大的地址块
方法：找这些地址的"最长公共前缀"

3.2 做题步骤

1. 把多个网络地址的二进制写出来（只需要关注不同的那个字节）
2. 从高位往低位看，找到第一个不同的位置
3. 前面全相同的就是公共前缀
4. 公共前缀长度 = 聚合后的 /n
5. 不同部分全部置0，得到聚合网络地址

3.3 例题

题目：

将以下4个网络聚合为一个路由：

192.168.0.0/24

192.168.1.0/24

192.168.2.0/24

192.168.3.0/24

一步步解：

前两组 192.168 完全相同，不需要看。

只看第三个字节（不同的部分）：

0 = 00000000

1 = 00000001

2 = 00000010

3 = 00000011

竖着看，从左边第几位开始不同？

第1位：都是0 ✓

第2位：都是0 ✓

第3位：都是0 ✓

第4位：都是0 ✓
第5位：都是0 ✓
第6位：都是0 ✓
第7位：0, 0, 1, 1 ← 这里开始不同！

前6位相同 → 这6位归网络号 → 后2位归主机号

原来每个是 /24，现在主机号多了2位（原来区分网络号的2位现在归主机号了）
→ 新前缀 = $24 - 2 = 22$

聚合网络地址：把后2位置0 → 00000000 = 0

答案：

聚合路由：192.168.0.0/22

验证：/22 总共能容纳 $2^{(32-22)} = 2^{10} = 1024$ 个地址，4个/24 总共 $4 \times 256 = 1024$ 个地址，刚好装下。✓

3.4 例题2

题目：

聚合：
172.16.16.0/24
172.16.17.0/24
172.16.18.0/24
172.16.19.0/24

答案：

16 = 00010000
17 = 00010001
18 = 00010010
19 = 00010011

前6位相同（000100），后2位不同。

→ $24 - 2 = 22$
→ 聚合为 172.16.16.0/22

4. IP层转发分组：最长前缀匹配

4.0 大白话理解

路由器收到一个数据包，往哪发？ 查路由表，看目的IP匹配路由表中的哪一条。

但一条IP可能同时匹配多条路由规则怎么办？ 比如 192.168.1.130 既匹配 192.168.1.0/24（范围128个地址），又匹配 192.168.1.128/25（范围64个地址）。这时候选**更精确的那条**（前缀更长的）——因为 /25 比 /24 描述得更精确。

直觉类比： 快递地址"浙江省杭州市西湖区XX路"。总部分拣时：

- "浙江方向" ✓ 匹配（很宽泛）
- "杭州方向" ✓ 匹配（精确一些）
- "西湖区方向" ✓ 匹配（最精确）

选最精确的那条路线走。这就是"最长前缀匹配"。

4.1 原则

匹配越具体（前缀越长），优先级越高。
默认路由 **0.0.0.0/0**（匹配一切）优先级最低——"实在不知道往哪走的，走这条"。

4.2 例题

路由表：

目的网络	下一跳/接口
192.168.1.0/24	接口0
192.168.1.128/25	接口1
192.168.0.0/16	接口2
0.0.0.0/0	接口3

题目：

目的IP为**192.168.1.130**，应从哪个接口转发？

分析：

逐条匹配：
192.168.1.0/24 → 范围 192.168.1.0~192.168.1.255，130在其中 ✓ 前缀/24
192.168.1.128/25 → 范围 192.168.1.128~192.168.1.255，130在其中 ✓ 前缀/25
192.168.0.0/16 → 范围 192.168.0.0~192.168.255.255，130在其中 ✓ 前缀/16

0.0.0.0/0

→ 匹配一切 ✓

前缀/0

最长前缀是 /25 → 选接口1

答案：

从接口1转发。

4.3 路由器的工作原理（简答题常考）

路由器做两件事：

路由（控制平面）：运行路由协议（RIP/OSPF/BGP），计算路径，生成路由表。

转发（数据平面）：收到包→查表→找到出口→送出去。

分组转发过程：

1. 输入端口收到IP数据报
2. 取出目的IP地址
3. 查转发表，最长前缀匹配找到输出端口/下一跳
4. TTL减1，重新计算首部校验和
5. 通过交换结构把分组送到输出端口
6. 输出端口排队，链路空闲时发送
7. 若没有匹配项，丢弃并可能返回ICMP差错报文

三代交换结构速记：

第一代（内存交换）：CPU参与，一次一个包——慢

第二代（总线交换）：共享总线，受总线带宽限制——快一些

第三代（交换矩阵/互连网络）：可并发转发多个包——最快

4.4 ICMP 网际控制报文协议

一句话：ICMP是IP层的"信使"，负责报告差错和传递诊断信息，不负责纠错。

ICMP 封装在 IP 数据报里传输。

ICMP 不是路由协议（RIP/OSPF/BGP才是）。

ICMP 只报告问题，不修复问题。

常见场景速查：

你用的命令	背后用的ICMP	原理
ping	Echo Request + Echo Reply	发个请求看对方回不回
tracert	TTL超时触发"时间超过"	逐步增加TTL，每跳收到超时报文
路由器无法转发	目的不可达报文	告知发送方"到不了"

5. RIP协议与路由表更新

5.0 大白话理解

RIP解决什么问题？ 每个路由器最初只知道"自己直连的网络在哪"。它需要知道"去往远方网络该走哪个邻居"。RIP的做法很简单：**每个路由器定期把自己的路由表发给所有邻居，邻居收到后把自己的距离加1，谁更近就选谁。**

直觉类比： 你刚搬到一个新小区，不知道最近的医院在哪。你问几个邻居，他们告诉你"去医院我要走10分钟"、"去医院我要走15分钟"、"去医院我要走5分钟"。你选了说5分钟的那个邻居——因为你跟着他走到医院就是 $5+1=6$ 分钟。

RIP的核心逻辑就是这样：

邻居说"我去X需要d跳" → 你跟邻居说"那我经过你去X需要d+1跳"
多个邻居都说能去X → 选跳数最小的

5.1 核心规则

RIP 是内部网关协议（IGP）
RIP 使用距离向量算法
度量值是跳数（经过几个路由器）
最大有效跳数 = 15
周期性向邻居交换整个路由表

— 用于一个自治系统内部
— 每个路由器只知道"方向+距离"
— RIP眼里跳数最少=最优
— 16表示"不可达"（RIP不适合大网络）
— 每30秒广播一次

5.2 路由表更新三规则

收到邻居 X 发来的路由信息后，对于每个目的网络 N：

规则1：「没有记录 → 直接加」
原来没有到N的路由 → 加入，距离=邻居距离+1，下一跳=X

规则2：「同一来源 → 必须更新」
到N的下一跳本来就是X → 不管新距离变大还是变小，都更新

规则3：「不同来源 → 择优更新」
到N的下一跳不是X → 只有当新距离更小才更新

为什么规则2是"必须更新"？ 因为邻居X是最了解"经过X到N"这条路径的人。就算X说距离变大了（比如它那边链路出问题了），你也要相信它——原来的旧数据已经不可靠了。

5.3 例题

路由器 R 当前路由表：

目的网络	距离	下一跳
Net1	2	A
Net2	5	B
Net3	4	C

R 收到邻居 A 发来的距离向量：

目的网络	A到该网络的距离
Net1	1
Net2	2
Net3	6
Net4	3

求更新后的路由表。

解：

先算出"如果我走A去每个网络，距离是多少"：

经A到Net1: $1+1 = 2$
经A到Net2: $2+1 = 3$
经A到Net3: $6+1 = 7$
经A到Net4: $3+1 = 4$

逐条用三规则判断：

Net1: 原下一跳就是A → 规则2 → 更新距离为2（实际上没变）
Net2: 原下一跳是B, 新距离3 < 原距离5 → 规则3 → 更新为3, 下一跳A
Net3: 原下一跳是C, 新距离7 > 原距离4 → 规则3 → 不更新, 保持原样
Net4: 原来没有 → 规则1 → 加入, 距离4, 下一跳A

答案:

目的网络	距离	下一跳
Net1	2	A
Net2	3	A
Net3	4	C
Net4	4	A

6. IP数据报分片

6.0 大白话理解

IP分片解决什么问题? 一个IP数据包从发送方到接收方, 途中可能经过不同类型的链路——以太网MTU=1500字节、WiFi可能更大、某些旧链路MTU更小。如果数据包大小超过了某段链路的MTU（最大传输单元），就需要切碎了运过去，到目的地再拼起来。

直觉类比: 你有一车家具要从北京运到深圳。全程高速公路没问题，但中间有一段窄巷子只能过小型三轮车。那就只能在进入巷子前把家具拆成几小车，过了巷子再重新组装。

原始大包 4000字节 → MTU=1500的链路 → 切成3片，每片≤1500字节

6.1 必背规则

标识 (Identification): 同一个原始报的所有分片，标识相同——用来认“你们是一家的”

MF (More Fragment): MF=1 后面还有分片，MF=0 这是最后一片

片偏移 (Fragment Offset): 本片数据在原始数据中的起始位置 / 8（以8字节为单位）

关键约束:

每个分片都有自己的IP首部（至少20字节）。

除最后一片外，每片的数据长度必须是8字节的整数倍。

片偏移以8字节为单位（所以偏移值 = 字节偏移 / 8）。

为什么片偏移要以8字节为单位？ IP首部中片偏移字段只有13位。如果用"字节"做单位，13位只能表示0~8191字节，而IP包最大可达65535字节，表示不了。改用8字节做单位后，13位可以表示 0~8191×8 = 0~65528 字节，刚好够用。这是工程上的精确设计。

6.2 例题

题目：

一个IP数据报总长度为4000字节，IP首部20字节。
要经过MTU=1500字节的链路。
求各分片的：数据长度、总长度、片偏移、MF标志。

一步步解：

Step 1: 原始数据部分 = 4000 - 20 = 3980字节

Step 2: 每片最大总长度 = MTU = 1500字节
每片最大数据长度 = 1500 - 20(IP头) = 1480字节

Step 3: 检查1480是否8字节的整数倍？
1480 ÷ 8 = 185，整除 ✓

Step 4: 切分：
第1片数据：1480字节（剩下 3980-1480=2500）
第2片数据：1480字节（剩下 2500-1480=1020）
第3片数据：1020字节（最后一片不要求是8的倍数）

Step 5: 计算各字段：
片偏移 = 本片数据前已有多少字节数据 / 8
第1片：偏移=0/8=0， MF=1（后面还有）
第2片：偏移=1480/8=185， MF=1（后面还有）
第3片：偏移=2960/8=370， MF=0（最后一片）

答案：

分片	数据长度	总长度(=数据+20)	片偏移	MF
1	1480	1500	0	1
2	1480	1500	185	1
3	1020	1040	370	0

★ Insight

注意"总长度" vs "数据长度"的区别:

- 总长度 = IP首部 + 数据部分 (每个分片都有自己的IP首部!)
- 数据长度 = 只有数据, 不含IP首部
- 片偏移基于"数据部分"的起始位置算, 不是"总长度"

考试容易错的地方: 片偏移到底填数字还是填"8字节的个数"?

填的是个数 (即字节偏移/8)。第2片数据从1480字节处开始→ $1480/8=185$ 。

7. 拥塞控制计算

7.0 大白话理解

拥塞控制解决什么问题? 发送方发得太快, 中间路由器处理不过来, 缓冲区溢出→丢包→重传→更多包→更堵。这是个恶性循环。拥塞控制就是让发送方"根据路况调整车速"——感知到拥堵就减速, 路况好了再慢慢加速。

直觉类比: 高速公路上的自适应巡航

你的车速 = cwnd (一次允许发送多少数据而不等确认)

路况好 = ACK按时回来 → 加速 (cwnd增加)

路况差 = ACK不回来/回来得很慢 → 减速 (cwnd减小)

慢开始: 刚上高速, 从1挡开始, 逐级翻倍加速 1→2→4→8...

拥塞避免: 到了一定速度后不再猛踩油门, 线性加速 16→17→18...

超时丢包: 前车突然急刹 (严重拥堵) → 车速直接降到1 (停车重新起步)

3重复ACK: 旁边车打双闪 (轻度拥堵) → 车速降一半继续走

7.1 拥塞控制 vs 流量控制 (简答必背)

很多初学者把这两个搞混。它们控制的是完全不同的东西:

	流量控制	拥塞控制
谁和谁	发送方 ↔ 接收方	发送方 ↔ 整个网络
目的是什么	防止 接收方 被撑爆	防止 网络中间路由器 被撑爆
怎么感知	接收方在ACK里明确告诉	发送方通过"丢包"间接推断
核心变量	rwnd (接收窗口)	cwnd (拥塞窗口)
谁说了算	接收方说了算	发送方自己猜

最终发送窗口 = $\min(rwnd, cwnd)$
(取两个窗口的最小值——既要照顾接收方承受能力, 也要照顾网络承受能力)

7.2 核心规则 (Reno版本——考试默认)

★ 先记住三个角色 ★

cwnd 拥塞窗口——"我现在一次最多发多少"
ssthresh 慢开始门限——"车速超过这个就不要再猛踩油门了"

★ 两个阶段 ★

慢开始: $cwnd < ssthresh$ 时 → 每经过1个RTT, **cwnd**翻倍 (指数增长)
拥塞避免: $cwnd \geq ssthresh$ 时 → 每经过1个RTT, **cwnd**加1 (线性增长)

★ 两个事件 ★

超时 (严重拥堵):
 ssthresh = $cwnd / 2$ (记住"拥堵阈值减半")
 cwnd = 1 (回到最慢, 重新慢开始)

3个重复ACK (轻度拥堵):
 ssthresh = $cwnd / 2$ (阈值减半)
 cwnd = **ssthresh** + 3 (不回到1! 在减半附近继续走)
 → 进入快速恢复阶段

为什么超时和3个重复ACK处理不一样?

这是TCP设计中最精妙的地方之一:

	超时	3个重复ACK
发生了什么事?	发出去的包和ACK都没回来	后面3个包都到了, 只缺1个
网络到底怎么了?	可能严重拥塞或断路	只是个别包丢了, 网络还在通
策略	重拳: cwnd 砍到1	轻拳: cwnd 减半
比喻	高速前方塌方	路上有个小坑

超时: 我喊了一声没人回应 → 彻底从头来
3重复ACK: 对方连喊"下一句! 下一句! 下一句!" → 我知道就那一句没听到, 补一句就行

Tahoe vs Reno (考试区分)

版本	超时	3个重复ACK	理念
TCP Tahoe	cwnd=1	cwnd=1	"丢包就是拥堵，一律重来"
TCP Reno	cwnd=1	cwnd减半+快恢复	"丢包分轻重，轻的不需要从头来"

考试默认按 Reno。如果题目明确写了 Tahoe，就按 Tahoe 来。

7.3 计算例题：基本变化

题目：

初始 cwnd=1, ssthresh=16。写出前7个RTT的cwnd值。

答案：

慢开始阶段 ($\text{cwnd} < \text{ssthresh}$)：

RTT0: cwnd = 1
 RTT1: cwnd = 2 (1×2)
 RTT2: cwnd = 4 (2×2)
 RTT3: cwnd = 8 (4×2)
 RTT4: cwnd = 16 (8×2, 到达门限！)

拥塞避免阶段 ($\text{cwnd} \geq \text{ssthresh}$)：

RTT5: cwnd = 17 (16+1)
 RTT6: cwnd = 18 (17+1)
 RTT7: cwnd = 19 (18+1)

★ Insight

注意 RTT4 的 cwnd=16:

- 它是在慢开始阶段通过翻倍到达的
- 恰好等于 ssthresh, 所以**下一个RTT**开始进入拥塞避免
- 有人会误以为 cwnd=16 时就切换, 其实是达到门限后"下一个RTT"才换挡

7.4 计算例题：发生超时

题目：

cwnd=24时发生超时，求新的ssthresh和cwnd。

答案：

```
ssthresh = 24 / 2 = 12
cwnd = 1          （超时重传，回到慢开始）
```

7.5 计算例题：3个重复ACK

题目：

cwnd=24时收到3个重复ACK（按Reno），求新的ssthresh和cwnd。

答案：

```
ssthresh = 24 / 2 = 12
cwnd = 12 + 3 = 15    （不减到1，而是在门限附近）
```

随后进入快速恢复：

每收到1个重复ACK → cwnd + 1

收到新数据ACK → cwnd = ssthresh = 12，退出快速恢复，回到拥塞避免

7.6 看图题答题模板

题目常给一张 cwnd 随时间变化的折线图，让你判断阶段和门限值。

读图四步：

1. 找"陡坡"（指数增长段）→ 这是慢开始
数字特征：1→2→4→8→16（每个RTT翻倍）
2. 找"缓坡"（线性增长段）→ 这是拥塞避免
数字特征：16→17→18→19（每个RTT加1）
3. 找"断崖掉到1" → 超时事件
新的 ssthresh = 掉落前cwnd / 2
4. 找"断崖掉一半（不掉到1）" → 3个重复ACK → 快速恢复
新的 ssthresh = 掉落前cwnd / 2
5. 慢开始→拥塞避免的转折点处的cwnd值 = 当时的ssthresh

标准答题模板：

该图中cwnd先呈指数增长（1→2→4→8...），说明处于慢开始阶段；
当cwnd达到ssthresh后转为线性增长（每个RTT+1），进入拥塞避免阶段。

图中cwnd突然降为1的位置，说明发生了超时，新的ssthresh=掉落前cwnd/2；
若cwnd只下降一半而未降到1，说明收到3个重复ACK，执行了快速重传和快速恢复。

7.7 易错点

易错点	正确理解
慢开始 = 增长慢？	慢是指起点低（从1开始），增长本身是指数的（翻倍），很快
cwnd≥ssthresh时怎么办？	进入 拥塞避免 ，不是回到慢开始。只有超时才回慢开始
超时和3重复ACK一样处理？	超时→cwnd=1（重拳）；3重复ACK→cwnd减半（轻拳）
发送窗口 = max？	取 $\min(rwnd, cwnd)$ ，是最小值
ssthresh是固定值？	每次超时或3重复ACK后都会更新为 $cwnd/2$
拥塞避免阶段还在翻倍？	拥塞避免是每RTT加1（线性），不是翻倍

7.8 网络层拥塞控制：RED与ECN（简答题）

前面讲的TCP拥塞控制是**端到端**的——发送方靠丢包"猜"网络拥堵。但路由器自己能看到缓冲队列在涨——这个信息不用就浪费了。

传统丢尾（Drop Tail）的问题

传统做法：缓冲区满了才丢包。这带来两个问题：

问题1（事后信号）：丢包时拥塞已经发生了，TCP要等超时才反应过来

问题2（全局同步）：多条TCP流共享一个路由器→同时满→同时丢→同时砍半→同时爬坡
→ 带宽利用率剧烈震荡："满→空→满→空"

RED（随机早期检测）

核心思想：不等满了再丢，缓冲区涨到阈值就开始"随机"丢。

为什么"随机"？为了打破全局同步——每次只让少数流感知到拥塞，其他流继续正常发，避免所有流一起降速造成的利用率塌方。

队列长度	行为
$< minth$	不丢
$minth \sim maxth$	以某种概率随机丢

队列长度	行为
> maxth	一定丢

ECN（显式拥塞通知）

比RED更进一步：不丢包，直接在包头做个"拥堵"标记，包继续传。

路由器检测到队列涨 → 在IP包头上标记CE=1 → 包继续走（不丢！）

→ 接收方看到标记 → 在TCP ACK中设置ECE标志

→ 发送方看到ECE → 降速（和丢包一样的减速效果，但没有真的丢数据包）

好处：不浪费已发送的数据，延迟敏感型应用（视频、游戏）尤其受益

	传统丢包	ECN
怎么传信号	丢了才知道（隐式）	包头标记（显式）
数据损失	需要重传	不丢包
反应速度	至少等1个超时	通过ACK即时带回
适合什么	所有流量	延迟敏感（视频/游戏/数据中心）

8. 信号带宽与数据率

8.0 大白话理解

物理层关心的是"一根线缆到底能传多快"。两个经典公式：

奈奎斯特：理想情况（无噪声），码元级别越多 → 传得越快

香农：现实情况（有噪声），信噪比越大 → 传得越快

直觉类比：

- 奈奎斯特 = 在完全安静的房间里，你每个字能多清晰地发音→对方能理解更多信息
- 香农 = 在嘈杂的餐厅里，你的声音必须盖过噪音→信噪比决定了信息传输上限

8.1 奈奎斯特公式（无噪声）

$$C = 2W \cdot \log_2(V)$$

C: 最大数据率 (bit/s)
W: 信道带宽 (Hz)
V: 每个码元可取的离散电平数 ("一个信号能表示几种状态")

例题:

带宽 $W=3000\text{Hz}$, 采用4种电平 ($V=4$), 求最大数据率。

$$\begin{aligned}C &= 2 \times 3000 \times \log_2(4) \\&= 6000 \times 2 \\&= 12000 \text{ bit/s}\end{aligned}$$

8.2 香农公式 (有噪声)

$$C = W \cdot \log_2(1 + S/N)$$

C: 最大数据率 (bit/s)
W: 信道带宽 (Hz)
S/N: 信噪比 (信号功率/噪声功率, 是比值不是dB)

dB换算 (考试必会):

$$\text{SNR(dB)} = 10 \cdot \log_{10}(S/N)$$

$$\text{反过来: } S/N = 10^{(\text{SNR}/10)}$$

常用值速记:

$$20\text{dB} \rightarrow S/N = 100$$

$$30\text{dB} \rightarrow S/N = 1000$$

$$40\text{dB} \rightarrow S/N = 10000$$

例题:

带宽 $W=3000\text{Hz}$, 信噪比30dB。求最大数据率。

$$30 = 10 \cdot \log_{10}(S/N) \rightarrow S/N = 10^3 = 1000$$

$$\begin{aligned}C &= 3000 \times \log_2(1+1000) \\&\approx 3000 \times \log_2(1001) \\&\approx 3000 \times 10 \quad (2^{10}=1024 \approx 1001, \text{ 所以 } \log_2 \approx 10) \\&\approx 30000 \text{ bit/s}\end{aligned}$$

★ Insight

$\log_2(1+S/N) \approx \log_2(S/N)$ 当 S/N 很大时。

$30\text{dB} \rightarrow S/N=1000 \rightarrow \log_2(1000) \approx 10$ (因为 $2^{10}=1024$)

所以 $30\text{dB} \approx$ 每Hz带宽传10bit, 这是考试速算的关键。

8.3 考不考?

先确认老师课上有没讲过奈奎斯特/香农。讲过就很可能考计算; 没讲过就不用管, 优先级远低于CRC、子网、拥塞控制。

9. 大题最后速背卡

CRC	补r个0 → 模2除 → 余数接后面
子网	块大小=256-掩码非255字节; 可用= $2^{\text{主机位}}-2$
路由聚合	找最长公共前缀, 不同位置0
IP转发	最长前缀匹配, $/25 > /24 > /16 >$ 默认
RIP	距离向量, 跳数 ≤ 15 , 16=不可达
RIP更新	邻居距离+1; 更短就更新; 同一来源必须更新
分片	MTU切片, 偏移 $\div 8$, MF最后为0
拥塞控制	慢开始翻倍, 避免+1; 超时 $\rightarrow 1$; 3ACK $\rightarrow$ 减半
RED/ECN	RED提前随机丢; ECN包头标记不丢包
带宽	奈奎斯特=无噪声; 香农=有噪声

10. 简答题专项 (背模板)

以下题目很像期末简答题原题。理解+背模板。

10.1 网络拓扑结构有哪些? 各有什么特点?

拓扑	一句话	优点	缺点
总线型	所有人共用一根线	结构简单、成本低	一根线坏了全网瘫
星型	所有人连到中心设备	管理方便、一户坏不影响别人	中心坏了全瘫
环型	大家手拉手围成圈	控制简单	一处断了全环废

拓扑	一句话	优点	缺点
树型	星型的层级扩展	便于扩展和管理	上层故障影响下层
网状	任意两点多条路	可靠性高、容错好	线太多、贵

答题模板：

计算机网络常见拓扑有总线型、星型、环型、树型和网状型。
总线型结构简单但单点故障影响大；星型便于管理但依赖中心设备；
环型控制简单但故障影响范围大；树型适合层次化扩展；
网状型可靠性最高但成本高。

10.2 电路交换 vs 分组交换

	电路交换	分组交换
建连？	通信前先建立专用电路	不需要预先建立
资源？	独占	共享（统计复用）
传送？	建连后连续传送	分成小包，逐跳存储转发
适用？	实时语音（时延稳定）	突发数据（利用效率高）

答题模板：

电路交换通信前建立端到端专用通路，通信中独占资源，结束后释放。
特点是时延稳定但资源利用率低（静默期也占着线路）。

分组交换把报文分成若干分组，每个分组独立经路由器存储转发，
多个用户共享链路。特点是资源利用率高，但可能出现排队时延、丢包和乱序。

10.3 交换机工作原理

交换机做三件事：**学习、过滤、转发。**

- 1. 收到帧 → 记下源MAC和入端口（学习）
- 2. 查目的MAC在哪个端口：
 - 查到了 → 只从那个端口转发（过滤，不打扰其他端口）
 - 查不到 → 向除入端口外的所有端口泛洪

为什么交换机可以全双工？ 交换机每个端口是独立冲突域，收发可同时进行。

为什么集线器只能半双工？ 集线器是物理层设备，所有端口共享一个冲突域——同一时间只能一个方向/一个设备发送，否则冲突。

10.4 停止等待协议

发送方：发包 → 等ACK → 收到ACK才发下一个 → 超时没收到就重传
使用序号来区分"新包"和"重传的旧包"

特点：实现简单，但信道利用率低（大部分时间在等待）

10.5 ARP协议解析过程

ARP = IP地址 → MAC地址的翻译官（只在局域网内工作）

1. 主机A想知道主机B的MAC地址（知道B的IP）
2. A先查自己的ARP缓存
3. 没有 → A在局域网内广播："谁是IP为X.X.X.X的人？请告诉我你的MAC地址"
4. 只有B回复（单播）："是我，我的MAC地址是XX:XX:XX:XX:XX:XX"
5. A把这条映射记入ARP缓存

10.6 NAT地址转换

NAT做的一件事：把内网IP:端口 ↔ 公网IP:端口的映射

1. 内网主机发出请求（源=192.168.1.5:3000）
2. NAT路由器改写为（源=公网IP:新端口，如1.2.3.4:5001），记录映射
3. 外部服务器回复给 1.2.3.4:5001
4. NAT查映射表，改写成 192.168.1.5:3000，转发进内网

10.7 TCP发送报文段的时机

TCP不是应用层写一次就发一次。以下时机触发发送：

1. 数据达到MSS（最大报文段长度）
2. 应用层要求立即发送（PUSH标志）
3. 发送方计时器超时
4. 收到对方ACK后，窗口允许继续发

10.8 路由器转发分组过程

1. 输入端口收到IP分组
2. 取出目的IP地址

3. 查路由表，最长前缀匹配找到下一跳
4. TTL减1，重新计算首部校验和
5. 通过交换结构送到输出端口
6. 链路层重封装，从输出接口发送
7. 无匹配且无默认路由 → 丢弃，可返回ICMP差错报文

10.9 浏览器访问网页的全过程（应用层+传输层）

【应用层视角】

用户在浏览器输入URL → DNS解析域名 → 浏览器构造HTTP请求报文
→ 发送给Web服务器 → 服务器返回HTTP响应（含状态码+网页内容）

【传输层视角】

HTTP基于TCP → 浏览器先与服务器三次握手建立连接
→ TCP提供可靠有序字节流传输HTTP报文
→ 传输完成后四次挥手释放连接
→ 若是HTTPS，TCP连接后还要做TLS握手

10.10 网络安全目标

目标	含义	常用实现手段
机密性	防止被偷看	加密
完整性	防止被篡改	摘要、MAC、数字签名
身份认证	确认对方是谁	证书、口令、公钥认证
不可否认性	发了不能赖账	数字签名
可用性	服务能正常用	备份、防火墙、抗DDoS

10.11 报文鉴别如何实现？

1. 报文摘要：对报文算一个"指纹"（定长输出）
2. MAC（消息认证码）：用共享密钥+摘要，双方用同一密钥验证
3. 数字签名：发送方私钥签名，接收方用公钥验证（还能实现不可否认）

核心逻辑：发送方生成摘要/签名 → 附在报文后 → 接收方重新计算比对

10.12 数字签名用的摘要算法应满足什么特性？

1. 定长输出：不管输入多长，输出固定长度
2. 单向性：从摘要无法反推原文
3. 抗碰撞：找不到两个不同输入产生相同摘要

4. 雪崩效应：原文改一丁点，摘要就面目全非
5. 计算高效：能快速算出摘要

数字签名流程：

发送方：报文 → 摘要算法 → 摘要 → 用私钥加密摘要 = 签名

接收方：用发送方公钥解密签名 → 得到摘要1

对报文重新算摘要 → 得到摘要2

摘要1 == 摘要2? 是 → 验证通过，且确认是发送方本人

10.13 典型IP计算题：192.168.1.100/26

题目：

IP地址：192.168.1.100/26

1. 属于哪一类地址？
2. 网络地址和广播地址？
3. 可用IP地址数？
4. 可用IP地址范围？

答案：

1. 第一个数是192 → C类地址

2. /26 掩码 = 255.255.255.192, 块大小 = $256 - 192 = 64$

子网：0~63, 64~127, 128~191, 192~255

100 落在 64~127

→ 网络地址：192.168.1.64

→ 广播地址：192.168.1.127

3. 主机位 = $32 - 26 = 6$

可用IP数 = $2^6 - 2 = 64 - 2 = 62$

4. 可用范围：192.168.1.65 ~ 192.168.1.126

附：零基础复习路线建议

如果你时间有限，按这个优先级顺序复习：

第一梯队（几乎必考，先拿下）：

- 子网掩码计算（网络号、广播地址、可用主机数）

- 拥塞控制计算（**cwnd**变化、超时/**3ACK**处理）
- **IP**数据报分片

第二梯队（高频）：

- **RIP**路由表更新
- 路由聚合
- **CRC**循环冗余校验
- 最长前缀匹配

第三梯队（简答题大概率考）：

- 电路交换 **vs** 分组交换
- 交换机工作原理
- 浏览器访问网页过程
- 网络安全目标与报文鉴别
- 路由器转发分组过程

第四梯队（看老师是否讲过）：

- 奈奎斯特/香农公式
- **TCP**发送报文时机
- **ARP/NAT**/停止等待协议